Lab Week 13 — Expressions, Functions & Iteration

Week 13 · Duration: 2 hours

Learning Objectives

By the end of this lab you will be able to:

- Use operators and the conditional (ternary) expression to make configuration environment-aware.
- Call built-in functions and explore them interactively with `terraform console`.
- Use `count` and `for_each` to create many resources from one block, and explain when to use each.
- Transform data with `for` expressions, including filtering and the splat operator.
- Generate repeated nested blocks with `dynamic` blocks.
- Handle uncertain values safely with `try` and `can`.
- Explain the module boundary: why the root module can only read what a child module declares as an output.

Before You Start

Checklist

- ☐ Terraform >= 1.16.3 (`terraform version`)

Starting point

```
cd ~/terraform-course
cp -r lab02 lab03
cd lab03
rm -rf .terraform .terraform.lock.hcl terraform.tfstate*
```

Part 1 — Explore Expressions in the Console (20 min)

`terraform console` is the fastest way to learn expressions. Nothing you type here touches AWS.

```
terraform init
terraform console
```

`terraform init` is required first: the console has to load the VPC module and the AWS provider before it can evaluate `var.*` and `local.*`.

Step 1.1 — Operators and conditionals

Type each of these and record the result:

```
> 5 + 3 * 2
> 10 % 3
> "a" == "a" && 1 < 2
> 3 > 5 ? "big" : "small"
> var.environment == "prod" ? "t3.large" : "t3.micro"
```

Step 1.2 — String functions

```
> upper("terraform")
> join("-", ["ipa", "lab03", "web"])
> split(",", "a,b,c")
> format("%s-%03d", "web", 7)
> substr("hello world", 0, 5)
> trimspace("  padded  ")
> replace("my_project_name", "_", "-")
```

Step 1.3 — Collection functions

```
> length(["a", "b", "c"])
> merge({a = 1}, {b = 2})
> keys({name = "kmitl", age = 60})
> values({name = "kmitl", age = 60})
> lookup({a = 1, b = 2}, "c", "default")
> contains(["dev", "prod"], "dev")
> toset(["a", "b", "a"])
> flatten([[1, 2], [3, 4]])
> zipmap(["a", "b"], [1, 2])
```

Step 1.4 — Regular expressions

```
> regex("[0-9]+", "instance-42-web")
> regexall("[a-z]+", "ab12cd34ef")
> can(regex("^t[23]\\.", "t3.micro"))
> can(regex("^t[23]\\.", "m5.large"))
```

Step 1.5 — Type conversion, try and can

```
> tostring(42)
> tonumber("3.14")
> tolist(toset(["b", "a"]))
> try(tonumber("not-a-number"), 0)
> can(tonumber("not-a-number"))
```

> **Checkpoint 1:** Record the output of every expression above in your report.
>
> **Question 1:** What is the difference between `try()` and `can()`? When would you reach for each?

Press Ctrl+D to exit the console.

Part 2 — Conditionals in Real Configuration (15 min)

Step 2.1 — Environment-driven sizing

In the root `locals.tf`, add the three locals below. Keep the `name_prefix`, `common_tags` and `is_production` entries you already have from Lab 2 — you are adding to the existing `locals` block, not replacing it.

```
locals {
  # ... name_prefix, common_tags and is_production stay as they are ...

  # Ternary: production gets a bigger instance
  effective_instance_type = local.is_production ? "t3.medium" : var.instance_type

  # Production gets a larger, encrypted root volume
  effective_volume_size = local.is_production ? 30 : var.root_volume.size_gb

  # Only open SSH to the office CIDR outside production
  ssh_cidrs = local.is_production ? [] : ["10.0.0.0/8"]
}
```

Step 2.2 — Verify without applying

```
terraform console
> local.effective_instance_type
> local.is_production
```

Then:

```
terraform console -var="environment=prod"
> local.effective_instance_type
```

> **Checkpoint 2:** Show both results side by side in your report.

Step 2.3 — Wire the first ternary into the module

A local that nothing reads is dead code, and `terraform validate` will not warn you about it. Consume `effective_volume_size` now. Add this local:

```
locals {
  # Root volume assembled from the ternary above. Production is always encrypted.
  effective_root_volume = {
    size_gb   = local.effective_volume_size
    type      = var.root_volume.type
    encrypted = local.is_production ? true : var.root_volume.encrypted
  }
}
```

Then, in the root `main.tf`, change the argument you pass to the module from the raw variable to the computed value:

```
module "web_server" {
  source = "./modules/web-server"

  # ... other arguments unchanged ...
```

```
  root_volume = local.effective_root_volume   # was: var.root_volume
}
```

The other two locals are consumed later: `effective_instance_type` in Step 3.4, once a map of servers exists to apply it to, and `ssh_cidrs` in Step 5.3, once the security group rules are data. That ordering is deliberate — each ternary meets the mechanism that needs it.

```
terraform fmt -recursive
terraform validate
```

Part 3 — count vs for_each (35 min)

This is the most important distinction in this lab.

Step 3.1 — Create instances with count

In `modules/web-server/variables.tf` add:

```
variable "instance_count" {
  type        = number
  description = "Number of identical web servers"
  default     = 2
}
```

In `modules/web-server/main.tf`, change the instance to the block below. This is the complete resource — note the four arguments carried over from Lab 2.

```
resource "aws_instance" "web" {
  count = var.instance_count

  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type

  # Carried over from Lab 2 - keep all three.
  subnet_id                   = var.subnet_id
  vpc_security_group_ids      = [aws_security_group.web.id]
  associate_public_ip_address = true

  # Carried over from Lab 2 - this is what consumes var.root_volume.
  root_block_device {
    volume_size = var.root_volume.size_gb
    volume_type = var.root_volume.type
    encrypted   = var.root_volume.encrypted
  }

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y nginx
    echo "<h1>Hello from Terraform - $(hostname)</h1>" >
/usr/share/nginx/html/index.html
    systemctl enable --now nginx
  EOF
```

```
  tags = merge(var.tags, {
    Name = "${var.name_prefix}-web-${count.index + 1}"
  })
}
```

Update the module outputs — with `count`, the resource becomes a list:

```
output "instance_ids" {
  value = aws_instance.web[*].id # splat expression
}

output "public_ips" {
  value = aws_instance.web[*].public_ip
}
```

Apply, then reduce `instance_count` from 2 to 1 and run `terraform plan`.

> **Question 2:** Which instance does Terraform destroy? Now imagine you had 3 instances and wanted to remove the middle one. What would `count` do? This is the classic `count` pitfall.

Step 3.2 — Now do it with for_each

Replace the `count` approach. First add the variable to `modules/web-server/variables.tf`:

```
variable "servers" {
  type = map(object({
    instance_type = string
    role          = string
  }))
  description = "Map of servers to create, keyed by logical name"
  default = {
    web = {
      instance_type = "t3.micro"
      role          = "frontend"
    }
    api = {
      instance_type = "t3.micro"
      role          = "backend"
    }
  }
}
```

Then replace the resource. Note three changes: the name is now `server`, `for_each` replaces `count`, and the size comes from each map entry rather than one module-wide variable.

```
resource "aws_instance" "server" {
  for_each = var.servers

  ami = data.aws_ami.amazon_linux.id

  # Per-server size, taken from the map
  instance_type = each.value.instance_type

  subnet_id                   = var.subnet_id
```

```
  vpc_security_group_ids      = [aws_security_group.web.id]
  associate_public_ip_address = true

  root_block_device {
    volume_size = var.root_volume.size_gb
    volume_type = var.root_volume.type
    encrypted   = var.root_volume.encrypted
  }

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y nginx
    echo "<h1>Hello from Terraform - $(hostname)</h1>" >
/usr/share/nginx/html/index.html
    systemctl enable --now nginx
  EOF

  tags = merge(var.tags, {
    Name = "${var.name_prefix}-${each.key}"
    Role = each.value.role
  })
}
```

With `for_each`, the resource becomes a map addressed by key:

```
output "server_ids" {
  value = { for k, v in aws_instance.server : k => v.id }
}
```

Step 3.3 — Remove the count-era leftovers

The `instance_ids` and `public_ips` outputs you wrote in Step 3.1 still refer to `aws_instance.web`, which no longer exists. Terraform fails with *"Reference to undeclared resource"* until you deal with them.

In `modules/web-server/outputs.tf`, delete both. In `modules/web-server/variables.tf`, delete the `instance_count` variable — `for_each` takes its length from the map.

```
terraform validate
```

This must pass before you continue. If it does not, you still have a reference to the old resource name somewhere.

Step 3.4 — Pass the map from the root

Right now the map lives only inside the module, so the root cannot see it and Part 4 will have nothing to read. Add the *same* variable to the root `variables.tf`:

```
variable "servers" {
  type = map(object({
    instance_type = string
    role          = string
```

```
  }))
  description = "Map of servers to create, keyed by logical name"
  default = {
    web = {
      instance_type = "t3.micro"
      role          = "frontend"
    }
    api = {
      instance_type = "t3.micro"
      role          = "backend"
    }
  }
}
```

Now consume `effective_instance_type` from Step 2.1. In `locals.tf`:

```
locals {
  # for + merge over a map of objects. In production every server is forced up
  # to local.effective_instance_type; otherwise each server keeps its own size.
  effective_servers = {
    for name, cfg in var.servers : name => merge(cfg, {
      instance_type = local.is_production ? local.effective_instance_type :
cfg.instance_type
    })
  }
}
```

And pass it down in the root `main.tf`:

```
module "web_server" {
  source = "./modules/web-server"

  # ... other arguments unchanged ...

  servers = local.effective_servers
}
```

Because the module now gets its size from each map entry, the module-wide `instance_type` variable is dead. Delete `variable "instance_type"` from `modules/web-server/variables.tf` and stop passing `instance_type` to the module from `main.tf`. Keep `var.instance_type` in the **root** — `local.effective_instance_type` still reads it.

Apply, then remove the `api` entry from the map and run `terraform plan`.

> **Checkpoint 3:** Compare the resource addresses: `aws_instance.web[0]` versus `aws_instance.server["web"]`.
>
> **Question 3:** Removing `api` from the `for_each` map destroys only the `api` instance. Explain in one sentence why `for_each` is safer than `count` for non-identical resources.

Part 4 — for Expressions (25 min)

Step 4.1 — Transform a list

In the console:

```
> [for s in ["web", "api", "db"] : upper(s)]
> [for s in ["web", "api", "db"] : "${local.name_prefix}-${s}"]
```

Step 4.2 — Filter with if

```
> [for k, v in var.servers : k if v.role == "backend"]
```

Step 4.3 — Build an object

```
> {for k, v in var.servers : k => v.instance_type}
```

Step 4.4 — Expose the instances from the module first

The root module cannot reach inside a child module. `module.web_server`. it is not a handle on the module's resources. So before the root can iterate over the instances, the module has to publish them.

In `modules/web-server/outputs.tf`, add:

```
output "instances" {
  description = "Instance details, keyed by the key from var.servers"
  value = {
    for name, inst in aws_instance.server : name => {
      id                = inst.id
      instance_type     = inst.instance_type
      private_ip        = inst.private_ip
      public_ip         = inst.public_ip
      public_dns        = inst.public_dns
      availability_zone = inst.availability_zone
    }
  }
}
```

Step 4.5 reads `public_dns` and `private_ip`, so both are listed here. Shaping the output like this is better practice than exposing `aws_instance.server` . The caller gets a documented contract instead of several hundred provider attributes that may change between provider versions.

Step 4.5 — Use for in real outputs

In the root `outputs.tf`:

```
output "server_summary" {
  description = "Map of server name to its public DNS"
  value = {
    for name, inst in module.web_server.instances :
```

```
    name => inst.public_dns
  }
}

output "frontend_servers" {
  description = "Names of frontend servers only"
  value = [
    for name, cfg in var.servers : name
    if cfg.role == "frontend"
  ]
}

output "all_private_ips" {
  description = "Splat over all instances"
  value       = values(module.web_server.instances)[*].private_ip
}
```

`values()` turns the map into a list so that `[*]` can splat across it — a splat cannot be applied to a map directly.

```
terraform fmt -recursive
terraform validate
```

> **Question 4:** Rewrite `[for k, v in var.servers : k if v.role == "backend"]` using only functions (`keys`, `lookup`, etc.). Which version is more readable?

Part 5 — dynamic Blocks (20 min)

Hard-coding repeated `ingress` blocks does not scale. `dynamic` generates them.

Step 5.1 — Define the rules as data

In `modules/web-server/variables.tf`:

```
variable "ingress_rules" {
  type = list(object({
    description = string
    port        = number
    protocol    = string
    cidr_blocks = list(string)
  }))
  description = "Security group ingress rules to generate"
  default = [
    {
      description = "HTTP"
      port        = 80
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    },
    {
      description = "HTTPS"
      port        = 443
      protocol    = "tcp"
```

```
      cidr_blocks = ["0.0.0.0/0"]
    }
  ]
}
```

Step 5.2 — Generate the blocks

```
resource "aws_security_group" "web" {
  name        = "${var.name_prefix}-web-sg"
  description = "Dynamically generated ingress rules"
  vpc_id      = var.vpc_id

  dynamic "ingress" {
    for_each = var.ingress_rules

    content {
      description = ingress.value.description
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(var.tags, { Name = "${var.name_prefix}-web-sg" })
}
```

A security group's `name` cannot be changed in place, so renaming it forces a replacement of the group — and of every instance that references it.

The module's `allowed_http_cidrs` variable is now unreachable: the rules arrive as `ingress_rules` instead. Delete `variable "allowed_http_cidrs"` from `modules/web-server/variables.tf` and stop passing it to the module. Keep it in the **root** — the next step uses it.

Step 5.3 — Add a third rule from the caller

The point of Step 5.1 is that rules are now *data*, so the caller can add one without the module changing at all. Build the list in the root `locals.tf`, and consume `ssh_cidrs` from Step 2.1 while you are there:

```
locals {
  candidate_ingress = [
    {
      description = "HTTP"
      port        = 80
      protocol    = "tcp"
      cidr_blocks = var.allowed_http_cidrs
```

```
    },
    {
      description = "Application port added by the caller"
      port        = 8080
      protocol    = "tcp"
      cidr_blocks = var.allowed_http_cidrs
    },
    {
      description = "SSH from the office network"
      port        = 22
      protocol    = "tcp"
      cidr_blocks = local.ssh_cidrs
    },
  ]

  # for ... if drops any rule with an empty CIDR list. local.ssh_cidrs is []
  # in production, so the SSH rule simply disappears there.
  ingress_rules = [
    for rule in local.candidate_ingress : rule
    if length(rule.cidr_blocks) > 0
  ]
}
```

Pass it to the module in the root `main.tf`:

```
module "web_server" {
  source = "./modules/web-server"

  # ... other arguments unchanged ...

  ingress_rules = local.ingress_rules
}
terraform plan
```

You should see three ingress rules: 80, 8080 and 22. The module file was not touched.

> **Checkpoint 4:** Show the plan diff proving the third ingress rule appeared.

Now run the same plan as production:

```
terraform plan -var="environment=prod"
```

Three things change at once, all from the ternaries you wrote in Part 2: the SSH rule disappears (`ssh_cidrs` is empty, so the `for ... if` filters it out), both instances become `t3.medium`, and the root volume grows to 30 GB encrypted.

> **Checkpoint 5:** Show the `dev` and `prod` plans side by side and point out all three differences.
>
> **Question 5:** In the `dynamic "ingress"` block, what do `ingress.key` and `ingress.value` refer to? What would they be if `for_each` were a map instead of a list?

Part 6 — Clean Up (5 min)

```
terraform destroy
terraform state list   # must be empty
\
```

Deliverables

Submit to Google Form:

- Complete root and module configuration
- `LabWeek13-report.md` containing:
- Checkpoints 1–5
- Answers to Questions 1–5